

AI Engineering Technical Challenge: The Multi-Modal Agentic Workflow

Role: AI Engineer (Internship)

Objective: To evaluate architectural intuition, proficiency with agentic frameworks (LangGraph), and full-stack integration capabilities.

1. The Mission

We are looking for engineers who can build systems that "think," not just chatbots that talk. Your task is to build a **Multi-Modal Travel Assistant** that aggregates data from different sources and renders it into a rich, interactive UI.

 **A Note on APIs:** If you do not have immediate access or subscriptions to live APIs (e.g., OpenWeatherMap, Unsplash, or advanced search tools), please construct **Mock APIs** using simple Python functions. These mock functions should simulate real-world latency and return structured, diverse, hard-coded data (e.g., JSON lists for the forecast, valid image URLs) to prove the agent's data processing and tool calling logic. The focus is on the agent's **architecture**, not the API key.

The User Story:

A user interacts with your Streamlit app and asks for a city (e.g., "Tell me about Kyoto"). The system must autonomously:

1. **Decide** where to fetch the information (Internal Database vs. Live Web Search).
2. **Fetch** current weather data and forecast.
3. **Retrieve** high-quality images of the location.
4. **Render** a combined response containing a text summary, a visual gallery, and an interactive data visualization.

2. Core Requirements (The "Must-Haves")

To pass this assignment, your submission must meet these technical baselines:

A. The Stack

- **Orchestration:** LangGraph (Required). We want to see a clear graph architecture (Nodes, Edges, State).

- **Frontend:** Streamlit. The output must be a GUI, not a console printout.
- **Models:** OpenAI (GPT-4o/Turbo) or Anthropic (Claude 3.5 Sonnet).

B. The "Switch" (Intelligent Routing)

Your agent must demonstrate **decision-making**. It cannot simply search the web for everything.

- **Vector Store Path:** Pre-populate a local vector store (ChromaDB, FAISS, or LanceDB) with detailed facts for 3 specific cities (e.g., Paris, Tokyo, New York).
- **Web Search Path:** If the user asks for a city *not* in your vector store (e.g., "Snohomish"), the agent must dynamically switch to a search tool (Tavily/DuckDuckGo or a **Mock Search**) to generate the answer.
- **Requirement:** Your graph must have a conditional edge that routes based on knowledge availability.

C. Structured Output

The agent must not stream raw Markdown to the UI. The final node of your graph must output a **Structured Object** (JSON/Pydantic) containing:

- city_summary: (String)
- weather_forecast: (List of data points for the next 5-7 days)
- image_urls: (List of strings)

The Streamlit app must parse this JSON to render a **Line Chart** (using st.line_chart or Plotly) alongside the text and images.

3. The "Extreme" Criteria (How to Stand Out)

Candidates aiming for the top percentile should attempt the following. These are not strictly mandatory but are heavily weighted in our evaluation.



Distinction 1: The "Manual" Transmission

Most engineers use high-level abstractions like create_tool_calling_agent or prebuilt.ToolNode.

- **The Challenge:** Build the tool execution logic **manually**. Define a custom node that parses the LLM's raw tool_calls payload, executes the function (live or **Mock**), and appends the ToolMessage back to the state.
- **Why:** This demonstrates you understand the raw API protocol of modern LLMs and aren't reliant on framework wrappers.

🏆 Distinction 2: Parallel "Fan-Out"

The weather API and the Image Search API are independent.

- **The Challenge:** Design your graph so these two fetch operations happen **in parallel** (asynchronous nodes) rather than sequentially, reducing overall latency.

🏆 Distinction 3: Human-in-the-Loop & Time Travel

- **The Challenge:** Implement LangGraph's **Memory (Checkpoint)**.
- If a user asks "Tokyo", and follows up with "What about next week?", the agent should understand the context (City = Tokyo) is preserved, but the date parameter has changed, and trigger *only* the weather tool update without re-fetching the city summary.

4. Submission Guidelines

Please submit a link to a private GitHub repository containing:

1. **Source Code:** Clean, modular Python code.
2. **graph.png:** A visualization of your LangGraph topology.
3. **README.md:** A short explanation of your architecture.

Evaluation Rubric

Component	Expectation
Architecture	Is the graph logic sound? Are edges defined correctly?
Code Quality	Is the state typed? Is the code modular?
UX/UI	Does the Streamlit app handle errors gracefully? (e.g., if Weather API fails)
"The Spark"	Did you attempt the Distinction Challenges?

Good luck. Build something beautiful.